

Provenance	T	$\mathbf{0}$	$\mathbf{1}$	$t_1 \oplus t_2$	$t_1 \otimes t_2$	$\ominus t$	$t_1 \ominus t_2$	$\tau(r_i)$	$\rho(t)$
diff-max-min-prob	$\mathbb{D}$	$\hat{\mathbf{0}}$	$\hat{\mathbf{1}}$	$\max(t_1, t_2)$	$\min(t_1, t_2)$	$\hat{\mathbf{1}} - t$	$t_1^{\text{fst}} == t_2^{\text{fst}}$	$(r_i, \vec{e}_i)$	t
diff-add-mult-prob	$\mathbb{D}$	$\hat{\mathbf{0}}$	$\hat{\mathbf{1}}$	$\text{clamp}(t_1 + t_2)$	$t_1 \cdot t_2$	$\hat{\mathbf{1}} - t$	true	$(r_i, \vec{e}_i)$	t
diff-top-k-proofs	Φ	$\emptyset$	$\{\emptyset\}$	$t_1 \vee_k t_2$	$t_1 \wedge_k t_2$	$\neg_k t$	$t_1 == t_2$	$\{\{\text{pos}(i)\}\}$	$\text{WMC}(t, \Gamma)$

Fig. 11. Definitions of three differentiable provenances.

$$\begin{aligned}
 \hat{a}_i &= (a_i, \nabla a_i) \in \mathbb{D} & \hat{a}_1 + \hat{a}_2 &= (a_1 + a_2, \nabla a_1 + \nabla a_2) & \min(\hat{a}_1, \hat{a}_2) &= \hat{a}_i, \text{ where } i = \text{argmin}_i(a_i) \\
 \hat{\mathbf{0}} &= (\mathbf{0}, \vec{\mathbf{0}}) & \hat{a}_1 \cdot \hat{a}_2 &= (a_1 \cdot a_2, a_2 \cdot \nabla a_1 + a_1 \cdot \nabla a_2) & \max(\hat{a}_1, \hat{a}_2) &= \hat{a}_i, \text{ where } i = \text{argmax}_i(a_i) \\
 \hat{\mathbf{1}} &= (\mathbf{1}, \vec{\mathbf{0}}) & -\hat{a}_1 &= (-a_1, -\nabla a_1) & \text{clamp}(\hat{a}_1) &= (\text{clamp}_0^1(a_1), \nabla a_1)
 \end{aligned}$$

 Fig. 12. Operations on dual-number $\mathbb{D} \triangleq [0, 1] \times \mathbb{R}^n$, where n is the number of input probabilities.

$$\begin{aligned}
 (\text{Variable}) \quad i &\in 1 \dots n & \varphi_1 \vee_k \varphi_2 &= \text{top}_k(\varphi_1 \cup \varphi_2) \\
 (\text{Literal}) \quad v &::= \text{pos}(i) \mid \text{neg}(i) & \varphi_1 \wedge_k \varphi_2 &= \text{top}_k(\{\eta \mid (\eta_1, \eta_2) \in \varphi_1 \times \varphi_2, \eta = \eta_1 \cup \eta_2, \eta \text{ no conflict}\}) \\
 (\text{Proof}) \quad \eta &::= \{v_1, v_2, \dots\} & \neg_k \varphi &= \text{top}_k(\text{cnf2dnf}(\{\{\neg v \mid v \in \eta\} \mid \eta \in \varphi\})) \\
 (\text{Formula}) \quad \Phi \ni \varphi &::= \{\eta_1, \eta_2, \dots\} & \Gamma(i) &= (r_i, \vec{e}_i)
 \end{aligned}$$

Fig. 13. Definitions used for diff-top-k-proofs provenance.

external interface for a provenance T . The whole execution pipeline is then illustrated below:



In the context of a Scallop application, an EDB is provided in the form $F_{\text{option}<I>}$. During the *tagging phase*, τ is applied to each input tag to obtain F_T , following which the SCLRAM program operates on F_T . For convenience, not all input facts need to be tagged—untagged input facts are simply associated by the tag $\mathbf{1}$ in F_T . In the *recovery phase*, ρ is applied to obtain F_O , the IDB that the whole pipeline returns. Scallop allows the user to specify a set of *output relations* and ρ is only applied to tags under such relations to avoid redundant computations.

4.5 Differentiable Reasoning with Provenance

We now elucidate how provenance also supports differentiable reasoning. Let all the probabilities in the EDB form a vector $\vec{r} \in \mathbb{R}^n$, and the probabilities in the resulting IDB form a vector $\vec{y} \in \mathbb{R}^m$. Differentiation concerns deriving output probabilities $\vec{y}$ as well as the derivative $\nabla \vec{y} = \frac{\partial \vec{y}}{\partial \vec{r}} \in \mathbb{R}^{m \times n}$.

In Scallop, one can obtain these using a *differentiable provenance* (DP). DPs share the same external interface—let the input tag space $I = [0, 1]$ and output tag space O be the space of *dual-numbers* $\mathbb{D}$ (Fig. 12). Now, each input tag $r_i \in [0, 1]$ is a probability, and each output tag $\hat{y}_j = (y_j, \nabla y_j)$ encapsulates the output probability y_j and its derivative w.r.t. inputs, ∇y_j . From here, we can obtain our expected output $\vec{y}$ and $\nabla \vec{y}$ by stacking together y_j -s and ∇y_j -s respectively.

Scallop provides 8 configurable built-in DPs with different empirical advantages in terms of runtime efficiency, reasoning granularity, and performance. In this section, we elaborate upon 3 simple but versatile DPs, whose definitions are shown in Fig. 11. In the following discussion, we use r_i to denote the i -th element of $\vec{r}$, where i is called a *variable* (ID). Vector $\vec{e}_i \in \mathbb{R}^n$ is the standard basis vector where all entries are 0 except the i -th entry.

4.5.1 diff-max-min-prob (dmmp). This provenance is the differentiable version of mmp. When obtaining r_i from an input tag, we transform it into a dual-number by attaching $\vec{e}_i$ as its derivative. Note that throughout the execution, the derivative will always have at most one entry being non-zero and, specifically, 1 or -1 . The saturation check is based on equality of the probability part only, so that the derivative does not affect termination. All of its operations can be implemented by algorithms with time complexity $O(1)$, making it extremely runtime-efficient.

4.5.2 diff-add-mult-prob (damp). This provenance has the same internal tag space, tagging function, and recover function as `dmmp`. As suggested by its name, its disjunction and conjunction operations are just $+$ and $\cdot$ for dual-numbers. When performing disjunction, we clamp the real part of the dual-number obtained from performing $+$, while keeping the derivative the same. The saturation function for `damp` is designed to always returns true to avoid non-termination. But this decision makes it less suitable for complex recursive programs. The time complexity of operations in `damp` is $O(n)$, which is slower than `dmmp` is but still very efficient in practice.

4.5.3 diff-top-k-proofs (dtkp). This provenance extends the *top-k proofs* semiring originally proposed in [Huang et al. 2021] to additionally support negation and aggregation. Shown in Fig. 11 and 13, the tags of `dtkp` are boolean formulas $\varphi \in \Phi$ in *disjunctive normal form* (DNF). Each conjunctive clause in the DNF is called a *proof* η . A formula can contain at most k proofs, where k is a tunable hyper-parameter. Throughout execution, boolean formulas are propagated with $\vee_k$, $\wedge_k$, and $\neg_k$, which resemble *or*, *and*, and *not* on DNF formulas. At the end of these computations, top_k is applied to keep only k proofs with the highest *proof probability*:

$$\Pr(\eta) = \prod_{\nu \in \eta} \Pr(\nu), \quad \Pr(\text{pos}(i)) = r_i, \quad \Pr(\text{neg}(i)) = 1 - r_i. \quad (1)$$

When merging two proofs during $\wedge_k$, there might be conflicting literals, e.g. $\text{pos}(i)$ and $\text{neg}(i)$, in which case we remove the whole proof. To take negation $\neg_k$ on φ , we first negate all the literals to obtain a *conjunctive normal form* (CNF) equivalent to $\neg\varphi$. Then we perform `cnf2dnf` operation (conflict check included) to convert it back to a DNF. To obtain the output dual-number $\hat{y}_j$ from a DNF formula φ_j , the tag for j -th output tuple, we adopt a differentiable *weighted-model-counting* (WMC) procedure [Manhaeve et al. 2021]. WMC computes the weight of a boolean formula φ_j given the weights of individual variables. Concretely, $\hat{y}_j = \text{WMC}(\varphi_j, \Gamma)$ where $\Gamma(i) = (r_i, \bar{e}_i)$ is the mapping from variables to their dual-numbers. Note that WMC is #P-complete, and is the main contributor to the runtime when using this provenance. The tunable k enables the user to balance between runtime and reasoning granularity. Detailed runtime analysis is shown in Appendix B.

4.6 Practical Considerations

We finally discuss some practical aspects concerning SCLRAM extensions and provenance selection.

Additional Features. We only presented the syntax and semantics of the core fragment of SCLRAM. SCLRAM additionally supports the following useful features: 1) sampling operations, and the provenance extension supporting them, 2) group-by aggregations, 3) tag-based early removal and its extension in provenance, and 4) mutually exclusive tags in `dtkp`. We formalize these in Appendix B.

Provenance Selection. A natural question is how to select a differentiable provenance for a given Scallop application. Based on our empirical evaluation in Section 6, `dtkp` is often the best performing one, and setting $k = 3$ is usually a good choice for both runtime efficiency and learning performance. This suggests that a user can start with `dtkp` as the default. In general, the provenance selection process is similar to the process of hyperparameter tuning common in machine learning.

5 IMPLEMENTATION

We implemented the core Scallop system in 45K LoC of Rust. The LoC of individual modules is shown in Table 1. Within the compiler, there are two levels of intermediate representations, front-IR and back-IR, between the surface language and the SCLRAM language. In front-IR, we perform analyses such as type inference and transformations such as desugaring. In back-IR, we generate query plans and apply optimizations. The runtime operates directly on SCLRAM and is based on semi-naive evaluation specialized for tagged

Module	LoC
Compiler	19K
Runtime	16K
Interpreter	2K
scallop	4K

Table 1. LoC of core Scallop modules.



Fig. 14. Visualization of benchmark tasks. Beside the name of each task we specify the size of the training dataset and the output domain. PacMan-Maze is omitted since it has been shown in Section 2.

semantics. There are *dynamic* and *static* runtimes for interpreted and compiled SCLRAM programs. Currently, all computation is on CPU only, but can be parallelized per-batch for machine learning.

Scallop can be used through different interfaces such as interpreter, compiler, and interactive terminal. It also provides language bindings such as *scallop* for Python and *scallop-wasm* for WebAssembly and JavaScript. With *scallop*, Scallop can be seamlessly integrated with machine learning frameworks such as PyTorch, wherein the *scallop* module is treated just like any other PyTorch module. When *jit* is specified in *scallop*, Scallop programs can be *just-in-time* (JIT) compiled to Rust, turned into binaries, and dynamically loaded into the Python environment.

Scallop provides 18 built-in provenances (4 for discrete reasoning, 6 for probabilistic, and 8 for differentiable). The WMC algorithm is implemented using *sentential decision diagram* (SDD) [Darwiche 2011] with naive bottom-up compilation. We allow each provenance to provide its own implementation for operations such as aggregation and sampling. This gives substantial freedom and enables optimizations for complex operations. Our provenance framework is also interfaced with *scallop*, allowing to quickly create and test new provenances in Python.

6 EVALUATION

We evaluate the Scallop language and framework on a benchmark suite comprising eight neurosymbolic applications. Our evaluation aims to answer the following research questions:

- RQ1** How expressive is Scallop for solving diverse neurosymbolic tasks?
- RQ2** How do Scallop's solutions compare to state-of-the-art baselines in terms of accuracy?
- RQ3** Is the differentiable reasoning module of Scallop runtime-efficient?
- RQ4** Is Scallop effective at improving generalizability, interpretability, and data-efficiency?
- RQ5** What are the failure modes of Scallop solutions and how can we mitigate them?

In the following sections, we first introduce the benchmark tasks and the chosen baselines for each task (Section 6.1). Then, we answer **RQ1** to **RQ5** in Section 6.2 to Section 6.6 respectively. All Scallop related and runtime related experiments were conducted on a machine with two 20-core Intel Xeon CPUs, four GeForce RTX 2080 Ti GPUs, and 768 GB RAM.

6.1 Benchmarks and Baselines

We present an overview of our benchmarks in Fig. 14. They cover a wide spectrum of tasks involving perception and reasoning. The input data modality ranges from images and videos to